W3 Python IoC Parser + Threat Intelligence

Log Parsing | JSON Reports | CSV Exports | Day 11-14

4 DAYS

Project Information

Project	Healthcare IoT Deception Honeypot Network
Week	Week 3 of 4 — Python Threat Intelligence Parser
Main File	logging/parsers/cowrie_parser.py
Output	data/processed/report_*.json + CSV files
Key Class	CowrieThreatIntelParser
Stack	Python + Pandas + Elasticsearch Client

Week 3 Objectives

- Write CowrieThreatIntelParser Python class to parse Cowrie JSON logs
- Extract IoC: attacker IPs, usernames, passwords, commands, malware hashes
- Generate JSON threat intelligence reports with top attacker rankings
- Export CSV files for all IoC categories
- Create final report script pulling 30-day summary from Elasticsearch

DAY 11 Parser Class — Core Code

This script is used to analyze Cowrie honeypot logs, extract important attack data (IP, username, password, commands, malware), generate a summary report, and export the results into JSON and CSV formats.

STEP 1 Create cowrie_parser.py — Full Class

Goal: Complete IoC parser

```
# Create file
cat > logging/parsers/cowrie_parser.py << 'PYEOF'
#!/usr/bin/env python3
import json, os, pandas as pd
from datetime import datetime, timedelta
from collections import Counter
```

```
class CowrieThreatIntelParser:
    def __init__(self, log_dir='./data/logs/cowrie', output_dir='./data/processed'):
        self.log_dir = log_dir
        self.output_dir = output_dir
        self.iocs =
{'ip_addresses':[],'usernames':[],'passwords':[],'commands':[],'malware_hashes':[]}
        os.makedirs(output_dir, exist_ok=True)

    def parse_logs(self, hours=24):
        cutoff = datetime.now() - timedelta(hours=hours)
        for fname in os.listdir(self.log_dir):
            if not fname.endswith('.json'): continue
            fpath = os.path.join(self.log_dir, fname)
            if datetime.fromtimestamp(os.path.getmtime(fpath)) < cutoff: continue
            with open(fpath) as f:
                for line in f:
                    try: self._extract(json.loads(line.strip()))
                    except: continue
        return self.iocs

    def _extract(self, e):
        event=e.get('eventid',''); src=e.get('src_ip'); ts=e.get('timestamp')
        if src: self.iocs['ip_addresses'].append({'ip':src,'ts':ts,'event':event})
        if 'login' in event:
            u=e.get('username',''); p=e.get('password','')
            if u: self.iocs['usernames'].append({'username':u,'ts':ts,'src_ip':src})
            if p: self.iocs['passwords'].append({'password':p,'ts':ts,'src_ip':src})
        elif event=='cowrie.command.input':
            cmd=e.get('input','')
            if cmd: self.iocs['commands'].append({'command':cmd,'ts':ts,'src_ip':src})
        elif event=='cowrie.session.file_download':
            h=e.get('shasum','')
            if h:
self.iocs['malware_hashes'].append({'hash':h,'filename':e.get('filename',''),'ts':ts,'src_ip':src
})

    def generate_report(self):
        ips=[i['ip'] for i in self.iocs['ip_addresses']]
        report={'generated_at':datetime.now().isoformat(),'summary':{
```

```
            'unique_ips':len(set(ips)),'unique_usernames':len(self.iocs['usernames']),

'unique_passwords':len(self.iocs['passwords']),'total_commands':len(self.iocs['commands']),

'malware_samples':len(self.iocs['malware_hashes'])},'top_attackers':Counter(ips).most_common(10)}

out=os.path.join(self.output_dir,f'report_{datetime.now().strftime("%Y%m%d_%H%M%S")}.json')
        with open(out,'w') as f: json.dump(report,f,indent=2)
        return report


    def export_csv(self):
        ts=datetime.now().strftime('%Y%m%d_%H%M%S')
        for key,fname in
[('ip_addresses',f'ips_{ts}.csv'),('usernames',f'usernames_{ts}.csv'),('passwords',f'passwords_{t
s}.csv')]:
            if self.iocs[key]:
pd.DataFrame(self.iocs[key]).to_csv(os.path.join(self.output_dir,fname),index=False)


if __name__=='__main__':
    p=CowrieThreatIntelParser()
    p.parse_logs(hours=24); r=p.generate_report()
    s=r['summary']
    print(f'Unique Attackers: {s["unique_ips"]}')
    print(f'Usernames Tried: {s["unique_usernames"]}')
    print(f'Commands: {s["total_commands"]}')
    p.export_csv()
PYEOF
```

DAY 12 Test + Run Parser

This step simulates brute force and command injection attacks on the honeypot and analyzes the generated logs using a parser to extract meaningful insights.

STEP 2	Install sshpass + Generate Attacks + Run Parser	Goal: Test data + parse

```
# Install sshpass
sudo apt install -y sshpass
# Generate attacks
for i in {1..10}; do
  sshpass -p 'root' ssh -p 2222 -o StrictHostKeyChecking=no \
```

```
    -o ConnectTimeout=3 root@localhost 'exit' 2>/dev/null || true
  echo "Attack $i done"; sleep 2
done
# Command injection
sshpass -p 'root' ssh -p 2222 -o StrictHostKeyChecking=no \
  root@localhost 'whoami; ls /; cat /etc/passwd; uname -a'
# Run parser
source venv/bin/activate
python3 logging/parsers/cowrie_parser.py
```

Expected Output:

```
Unique Attackers: 1
Usernames Tried: 6
CSV exports done
```

DAY 13 Final Report Script

This step creates and executes a Python script to analyze Elasticsearch data and generate a final report summarizing total attacks and unique attackers.

STEP 3	Create + Run generate_final_report.py	Goal: 30-day ES summary

```
# Create script
cat > analysis/reports/generate_final_report.py << 'PYEOF'
import json,os
from datetime import datetime
from elasticsearch import Elasticsearch

es=Elasticsearch(['http://localhost:9200'])
report={'project':'Healthcare IoT Deception Honeypot','generated_at':datetime.now().isoformat(),
    'recommendations':['1. Network segmentation','2. Disable default credentials',
    '3. Block top IPs at firewall','4. Enforce MFA']}
try:
    r=es.search(index='honeypot-*',body={'query':{'range':{'@timestamp':{'gte':'now-
30d'}}},'size':0,

'aggs':{'total':{'value_count':{'field':'src_ip.keyword'}},'unique':{'cardinality':{'field':'src_
ip.keyword'}}}})
    a=r['aggregations']

report['summary']={'total_attacks':a['total']['value'],'unique_attackers':a['unique']['value']}
```

```
except Exception as e: report['summary']={'error':str(e)}
fname=f'./analysis/reports/final_report_{datetime.now().strftime("%Y%m%d_%H%M%S")}.json'
with open(fname,'w') as f: json.dump(report,f,indent=2)
print(json.dumps(report,indent=2))
PYEOF
```

```
# Run it
python3 analysis/reports/generate_final_report.py
```

Expected Output:

```
{"summary": {"total_attacks": 95, "unique_attackers": 10}}
```

DAY 14 Data Analysis + Test Events

These commands are used to manually insert simulated attack logs into Elasticsearch for testing and verifying the ELK stack visualization and analysis

STEP 4 Add Different Attack Types Manually

Goal: Populate all dashboard categories

```
# Use attacks alias (from Week 1)
attacks
# Ya manually:
```

```
# command_injection
curl -X POST "http://localhost:9200/honeypot-$(date +%Y.%m.%d)/_doc" \
  -H 'Content-Type: application/json' \
  -d '{"@timestamp":"'$(date -u +%Y-%m-%dT%H:%M:%S.000Z)'",
  "attack_type":"command_injection",
  "src_ip":"8.8.0.1","eventid":"cowrie.command.input"}'
```

```
# credential_theft
curl -X POST "http://localhost:9200/honeypot-$(date +%Y.%m.%d)/_doc" \
  -H 'Content-Type: application/json' \
  -d '{"@timestamp":"'$(date -u +%Y-%m-%dT%H:%M:%S.000Z)'",
  "attack_type":"credential_theft",
  "src_ip":"5.8.0.1","eventid":"cowrie.login.success"}'
```

NOTE: 'attacks' alias use karo — automatically sab types generate karta hai

Completion Checklist

- ✓ logging/parsers/cowrie_parser.py created with full class
- ✓ python3 logging/parsers/cowrie_parser.py runs without errors
- ✓ data/processed/report_*.json generated with summary
- ✓ data/processed/ contains ips_*.csv, usernames_*.csv, passwords_*.csv
- ✓ generate_final_report.py created and runs correctly
- ✓ Kibana shows 3+ attack types: bruteforce, command_injection, credential_theft
- ✓ attacks alias works: generates test data automatically

Week 3 — Required GitHub Commits

Commit Message	Files Changed
`git commit -m "feat: add CowrieThreatIntelParser class"`	`logging/parsers/cowrie_parser.py`
`git commit -m "feat: add parse_logs() and _extract() methods"`	`logging/parsers/cowrie_parser.py`
`git commit -m "feat: add generate_report() JSON report method"`	`logging/parsers/cowrie_parser.py`
`git commit -m "feat: add export_csv() for IoC data export"`	`logging/parsers/cowrie_parser.py`
`git commit -m "feat: add generate_final_report.py script"`	`analysis/reports/generate_final_report.py`
`git commit -m "feat: add auto-attacks.sh for test data generation"`	`~/auto-attacks.sh`
`git commit -m "test: verify parser extracts 6+ unique usernames"`	`data/processed/report_*.json`
`git commit -m "test: verify CSV files generated in data/processed/"`	`data/processed/ips_*.csv, usernames_*.csv`